

GRACE

Review of Palak's working design — building it in Django, and what that changes

Prepared by: Stephen Gadd · **Date:** 25 August 2026 · **For:** Ruth Mostern, Palak Vashist, Alexandra Straub · **Purpose:** response to "Tracking people, projects, and datasets — a working design for our editorial workflow"

Palak's draft describes a **coherent system**. The entity graph in the prose and the diagram agree with each other, the cardinalities are stated and correct, and the "how is it controlled / what values are allowed" discipline that Ruth asked for has produced exactly the data dictionary a build needs. This note proposes **four structural fixes**, answers the five questions Palak left open, and raises **one governance point** nobody has yet put on the table.

The system is called **GRACE** — *Gazetteer Register And Contact Engagement* — and **where it is being built has changed**. Baserow was a false start and we are not going back to it: GRACE will be a **Django app inside WHG**, alongside **datasets**, **collection** and the rest. That decision does more to this design than it might appear, and mostly in its favour — several of Palak's open questions are answered outright by things WHG already has. It also carries one real cost, addressed in §5. The loose ends left behind by the Baserow retreat are listed in §10.

A note on terms. Every contribution to WHG is a **gazetteer**, the single term replacing the older "dataset" / "collection" split. Today a gazetteer is realised in the software as a *Dataset* or a *Collection*; in future the concept extends to routes, itineraries and networks. A *print historical gazetteer* is a **source** we might turn *into* a WHG gazetteer — a lead, not a gazetteer in the platform sense. §7 explains why the draft's persistent "dataset" should not simply be find-and-replaced, and what to do instead.

1. What is working, and should not be touched

- **"Registers, not phases."** **KEEP** This is right, and it is the insight the June sketch lacked. Work genuinely does not move in a line — a gazetteer can be in live correspondence with its owner *and* halfway through review. Naming them registers, in the old sense of a running official list, buys us out of a false sequence.
- **Two questions per field.** **KEEP** "How is it controlled" and "what values may it hold", answered for every field, is the difference between a spreadsheet and a system. The four-way classification (pick-list / date / link / free text) maps cleanly onto Django: pick-list becomes a lookup table or **choices**, link becomes a **ForeignKey**, and the discipline survives the change of platform intact.
- **An engagement is with a person, not with a record.** **KEEP** This is the single best modelling decision in the draft. It is what makes "open one person and see every conversation" possible, and it is why the register survives a project being renamed or a gazetteer being withdrawn.
- **Action items as real tasks.** **KEEP** Palak's own correction — from a free-text plan to an owned, dated task — is exactly right, and it is what makes the reminder rule work at all.
- **Print gazetteers folded into Sources.** **KEEP** Giving them their own record would have duplicated the bibliography for no gain. One source type is the simpler and correct call.

2. Fix 1 — Register 3 should point at WHG's state, not restate it

The proposed *Dataset tracking status* — on our radar; in internal review; revisions requested; ready to publish; in external peer review; published; indexed; not indexed; on hold; declined — conflates two axes. Some of those values are editorial judgements. Others are machine facts WHG already owns, in two places: `Dataset.ds_status` (seed → format_ok → uploaded → reconciling → wd-complete → accessioning → indexed) and `GazetteerRegistryEntry.status` (draft / submitted / rejected / pending / embargoed / published), alongside a separate re-ingestion status. Palak flags the problem herself — “the dashboard stages are guesses... our technical director will know how they line up”.

Building in Django turns this from a synchronisation problem into a modelling one, which is much easier. Nothing needs copying, protecting or reconciling: the tracker record simply *points* at the real thing and reads its state live. The shape that does this — and that solves the prospect-versus-actual problem in §7 at the same time — is a nullable foreign key:

```
class TrackedGazetteer(models.Model):
    # The editorial record. Exists from the moment something lands on our radar,
    # long before there is anything in WHG to point at.
    dataset = models.ForeignKey('datasets.Dataset', null=True, blank=True, ...)
    stage = models.ForeignKey('Stage', ...) # editorial values ONLY
    owner = models.ForeignKey(settings.AUTH_USER_MODEL, ...)

    @property
    def indexed(self):
        # never stored, never stale
        return self.dataset and self.dataset.ds_status == 'indexed'
```

The consequence is worth stating plainly: **every machine fact disappears from the tracker's own fields**. Published and indexed state, date indexed, the link into WHG, record count, temporal extent, spatial coverage, licence, rights holder, contributors, and which authorities a gazetteer was reconciled against are all read through the foreign key. They cannot drift, because there is only ever one copy. What remains in the tracker is what is genuinely ours to decide: the editorial stage, the responsible person, review status, permission to publish, when it came onto our radar, and the notes.

Recommendation: delete the machine values (*published*, *indexed*, *not indexed*) from the editorial stage list — they are derived, not chosen — and model the tracker record with a nullable FK to `Dataset` so it can exist before the gazetteer does. Rename the register (see §8) so it stops colliding with WHG's contributor dashboard.

Decision needed: confirm the editorial stage list once the derived values are removed. That is Palak's to define; the derived half is no longer a question.

3. Fix 2 — half of the Catalog is already built

This is the largest practical change from building in-house, and it is good news. Two of Palak's tables substantially exist already, and re-creating them would give us two competing copies of the same facts.

People is mostly `users.User`

The existing user model already carries `name`, `given_name`, `surname`, `affiliation`, `web_page`, `orcid`, `email`, `email_confirmed`, `role`, `is_active`, `preferred_language` — and `news_permitted`, which is

precisely the mailing-list consent flag the draft's *Mailing lists* field implies. Palak's *Identifier* field is `orcid`; her *Status: active / dormant* is `is_active`.

The catch is that most people in the Catalog will never have a WHG account — we have merely heard of them — and we must not create shell accounts for them. So the right shape is a separate `Contact` model with an optional one-to-one link to `User`, and a firm rule that no field is stored in both places: where a `Contact` is linked to a `User`, affiliation, ORCID and email are read *through* the link, never duplicated. This also turns Palak's own aside — "*ultimately the people submitting will become the user*" — from a note in the margin into a real, first-class relation.

Regions is already M49, and already built

Palak calls her region list a placeholder and asks for "a real scheme". WHG has one: `regions.Region` is a complete UN M49 hierarchy — `global / region / sub-region / intermediate / country` — with parent and member relations, ISO alpha-2 and alpha-3 codes, Wikidata identifiers, multilingual labels, and actual geometry and hulls. The placeholder list can simply be deleted in favour of a many-to-many to `Region`, which also answers her question about multiple regions: yes, and it comes for free.

Recommendation: do not build a `People` table from scratch — build `Contact` with an optional link to `User` and no duplicated fields. Delete the placeholder region vocabulary and use `regions.Region` directly.

4. Fix 3 — two entities are still missing

Organisations

Affiliation is free text, and there is no institution record anywhere in the design — nor is there one in WHG, so this genuinely is new. Permission to publish is granted by an archive, a library, a museum or a university, not by an individual, and *Permission to publish* is the field the entire licensing programme depends on. Projects are not institutions: a project ends, an institution does not, and the rights it granted outlive both the project and whoever we spoke to. A minimal `Organisations` table also gives us somewhere to record that we already hold a standing agreement with an institution before we approach a third researcher there.

Intake and triage

There is nowhere for a public submission to land. *Discovery source* offers `conference; referral; literature; flagged internally; other` and no value for a web submission, and nothing in the three registers says "this arrived from the public and nobody has looked at it yet". This matters more now than it did under Baserow, because the public front door at `/contribute/` currently redirects to the Baserow form (§10) and will have to be rebuilt in Django regardless — so the intake model and the form should be designed together rather than one retrofitted to the other.

Related: *Sources* → *Associated datasets* is documented as "datasets it documents". That is a **documentation** relation. There is no **derivation** relation — the candidate gazetteer that was extracted *from* a given print source. Since turning print gazetteers into WHG gazetteers is the main reason the bibliography exists, losing that link would cost us the provenance chain for the whole programme.

Recommendation: add an `Organisations` model; add `web form` to *Discovery source* and an *unreviewed* intake state; and split the *Source* → gazetteer link into "documents" and "derived from".

Decision needed: the question left open since June — is the public form a *suggest-a-source* tool, or a general front door that triages everyone and routes gazetteer owners on to the validate flow? The Django rebuild of that form waits on the answer.

5. Fix 4 — keeping Palak's self-service, which is what Django threatens

The reason the in-house approach was set aside in June was not technical. It was that a no-code tool lets Palak add a field or a vocabulary term herself, without a developer, and Django ordinarily does not. If we build this in-house and ignore that, we will have solved the data-modelling problem and recreated the bottleneck the team was trying to escape.

The mitigation is a deliberate modelling choice, and it should be made at the start rather than retrofitted: **model each controlled vocabulary as its own table, not as a `choices=` tuple.** A `choices` list is frozen in code and needs a developer and a migration to change; a lookup table is editable in the Django admin in ten seconds. Reserve `choices` for the handful of lists that code actually branches on — where adding a value without changing logic would be a bug — and make everything else a table Palak owns.

Applied to the draft's vocabulary list, that means *People roles*, *mailing lists*, *project status*, *source type*, *priority*, *contact status*, *interaction channel*, *action-item status*, *review recommendation*, *outcome*, *discovery source*, *digitization status*, *content type* and *content status* are all tables. Only the tracker's editorial stage is a plausible candidate for `choices`, and only if notifications branch on specific values.

Recommendation: vocabularies as tables by default, with an admin group giving Palak edit rights over them. Agree the short list of code-critical vocabularies that stay frozen, and say so in the model docstrings so nobody later "tidies" them into `choices`.

6. Field-level corrections

Field	As drafted	Change to	Why
People · Name, affiliation, ORCID, email	Free text, re-declared	Read through the optional <code>User</code> link	§3 — never store the same fact twice
People · Mailing lists	Pick-list incl. <code>none</code>	Reconcile with <code>User.news_permitted</code>	Consent already has a home; two flags will disagree
Region (projects & gazetteers)	Single pick-list, placeholder values	M2M to <code>regions.Region</code>	M49 with geometry, ISO codes and labels is already built
Gazetteer · Licence / rights	Hand-written list of 8 values	FK to <code>licensing.License</code>	The canonical licence vocabulary, already used by the public licences page and the Workbench picker
Gazetteer · Record count	Free text	Derived from the linked Dataset	§2 — not ours to type
Gazetteer · Time period	Free text, e.g. <code>900-1200</code>	Keep the prose, add numeric start/end year	Unsortable and unfilterable as text; WHG models temporal extent numerically

Gazetteer · Language(s)	Free text	ISO 639-3, multiple	The data is multilingual; free text will not group
Register 3 · Reconciled against	Wikidata; TGN; GeoNames; other; none	Derived from the registry	We index considerably more than four authorities, and the set changes
Discovery source	No web-form value	Add <code>web form</code>	§4
Interactions · Logged by	Link to one of us	Drop	Duplicates the universal <i>added by</i> the draft says to assume on every record
Gazetteer · Status	Absent	Consider adding	People and Projects both carry one; the asymmetry is probably unintended

Three structural nits

- **There are four registers, not three.** Content is not engagement — it is output. The draft half-admits it (*"content ideas: quick jots — planned pieces live in Content"*). Either promote it, or drop the claim that each register is one working table; it is already untrue of the Catalog, which is four.
- **Two Responsible person fields can disagree.** One sits on the Engagement, one on Register 3, with no rule about which wins. Proposed rule: one accountable owner per gazetteer, inherited by its engagements unless deliberately overridden.
- **The notification rule misses the failure mode that matters.** "Stage change notifies the responsible person" is a good, restrained rule — but what actually goes wrong in outreach is *stalling*, and a stall is an *absence* of a stage change. Nothing fires when an engagement has sat in *awaiting reply* for six weeks. The fix is nearly free: make *Next follow-up* mandatory whenever the stage is an open one, and the existing date reminder becomes a staleness alarm. Worth suppressing the notification when the responsible person is the one who made the change — otherwise the system spends its credibility telling people what they just did. Building in Django means these ride the existing `whgmail` machinery and Celery Beat rather than needing anything new.

7. The five questions Palak left open

Question	Answer
1. Regions — placeholder list; can something sit in more than one?	Already solved, and yes. <code>regions.Region</code> is a full UN M49 hierarchy with geometry, ISO codes, Wikidata ids and multilingual labels. Use a many-to-many to it and the multi-region question answers itself; single-region is simply the common case, not a constraint worth encoding.
2. Priority and stage — on the engagement, or on the catalog record?	The register logic answers it. A stage describing <i>the conversation</i> belongs on the Engagement; a stage describing <i>the thing</i> belongs on the thing. Contact status stays where it is (correct as drawn); the editorial stage belongs to the tracked gazetteer. Priority is a property of the conversation: leave it on the Engagement.
3. Funding and grants — right home? Does it belong at all?	It belongs, for a reason the draft could not have known. Funder and grant number are part of the <i>credit chain</i> WHG already models — the gazetteer registry carries CRediT contributors, rights holder and structured citation text. Keep <i>Funder</i> and <i>Grant number</i> on Projects, and note that they feed citation metadata. That is the justification, and it needs no new scope.

4. Dashboard stages — guesses, to be set with the technical director.

See §2. The derived half of that list is no longer a question — it is read through the foreign key. What remains is the genuinely editorial half, which is Palak's to define and needs no technical input at all.

5. External peer review — how much to build now?

None of it now — the draft is right to sketch and defer. Two notes for when we do: WHG's record-suggestion flow already runs *pending* / *approved* / *rejected* with a reviewer, timestamp and review note, and *embargoed* already expresses "accepted but not yet public". Building in-house also means review comments could reuse the existing comment model rather than inventing a parallel one.

8. On "dataset" and "gazetteer"

The draft uses "dataset" throughout, and it is worth being precise about why a find-and-replace would be the wrong correction. Palak's usage is not careless — it matches the current Django model name, which is exactly why it reads naturally.

The real issue is that the table holds *two different kinds of thing*: prospects (format `unknown`, permission `not yet asked`, someone mentioned it at a conference) and actual WHG gazetteers. "Dataset" is wrong for the second; "gazetteer" is premature for the first.

Recommendation: rename the table `Gazetteers` and let the nullable `Dataset` link from §2 carry the prospective-versus-actual distinction structurally — a row with no linked `Dataset` is a prospect, and no vocabulary has to say so. This is the same idea `GazetteerRegistryEntry.status` already expresses in running from *draft* to *published*.

Two riders. *Source type*: *gazetteer* stays as it is, but should be labelled "**printed gazetteer**" — that is the one place in the whole system where the word means a *lead* rather than a contribution, and the label should say so. And the code will keep the `Dataset` model name for now: this is a change to human-facing language, not a migration.

9. Naming

The registers

Within GRACE, "Dashboard" has to go. WHG's dashboard is the logged-in contributor's own workspace — their gazetteers, their uploads — and this is a staff editorial queue with a different audience entirely. Now that both live in the same Django project the collision would be literal, not just conversational. Palak asks whether the two might merge; they should not, though her underlying instinct is sound and is answered by the `User` link in §3. Proposed: **Catalogue · Engagement · Pipeline**.

The system — settled

The system is **GRACE: Gazetteer Register And Contact Engagement**. It expands onto exactly what the system does, it is pronounceable in a meeting, and it centres *people* — the honest description of something that is mostly about relationships. It also works as a Django app label: `grace/` sits comfortably beside `datasets` and `collection`, and it collides with nothing we already use.

Also considered and set aside: *Periplus* (an ancient coastal itinerary — evocative, but it needs a gloss every time it meets someone new), *GEM* (Gazetteer Engagement Manager — plainer, but it says less), and *Prospect* (the best conceptual fit, since a prospect is both a lead and a view over a landscape, but eight letters was a stretch to hold together). **Atlas** and **Workbench** were ruled out as already taken.

10. Personal data — sharper than it looks

Building in-house improves this considerably: the data stays at Pitt, under our own auth, backups and access control, with no third-party processor and no export to argue about. But it does not dispose of the question, and one detail makes it more pressing rather than less.

WHG encrypts user emails at rest. `User.email` is an `EncryptedTextField`, with an indexed HMAC (`email_hash`) carrying equality lookups precisely because the column itself is unqueryable. A new Contact table storing plain-text addresses for people who never signed up would hold *less*-protected personal data than we hold for people who did — a visible regression against the platform's own standard, in the same database. Whatever we build must meet the bar the user model already sets.

The rest stands as before: the Catalogue will hold names, addresses, affiliations and mailing-list membership for researchers who never asked to be in a database. There is no recorded lawful basis, no consent capture beyond `news_permitted`, and no deletion path — *dormant* is a status, not erasure. And because contacts and account-holders will now sit in one system, a subject access request spans both, which is an argument for settling the retention and erasure rules while the models are still on paper.

Decision needed: Ruth to confirm the lawful basis for holding contacts who never opted in, and the retention and erasure rule. Technical follow-on: match the encryption standard set by `users.User`.

11. Loose ends from the Baserow retreat

Dropping Baserow leaves live code and content pointing at it. None of it is urgent except the first, which is a public front door.

Loose end	What to do
The public form <code>main/</code> <code>views.py::submit_dataset</code> , <code>/</code> <code>contribute/</code> , "Suggest a Source" in the Data menu	Currently a 302 to the external Baserow form — the site's public entry point for suggestions now leads to a system we are abandoning. Rebuild as a Django form writing to the intake model in §4. Design the two together.
The licence sync <code>sync_licences_to_baserow</code>	Retire. Its purpose was to keep an external copy of the licence vocabulary in step; an FK to <code>licensing.License</code> makes the copy unnecessary.
Settings and credentials <code>BASEROW_*</code> in <code>whg/settings.py</code>	Remove once nothing reads them. The bot credentials and API token should be revoked at Baserow rather than merely unset — the token was circulated by email in the clear.
The bibliography	Currently held in Baserow's <i>Gazetteer Bibliography</i> table, itself loaded from <code>WHG_gazetteer_bibliography.xlsx</code> . It needs exporting and importing into Sources — and it is the obvious first real content for the new app.
The retired <code>leads</code> prototype	The earlier in-Django attempt (a lead model, an admin triage UI, a public form) exists on the <code>atlas</code> branch. Worth reading before starting: some of it is close to what §4 needs. Note that the teardown SQL recorded in <code>developer/baserow-workflow-tool.md</code> should not now be run.
The Baserow design note <code>developer/baserow-workflow-tool.md</code>	Mark superseded rather than deleting — it records why the decision was taken and then reversed, which is worth keeping.

Summary — decisions to settle

Decision	The question	My recommendation
1. Contacts vs users	One <code>Contact</code> model linked to <code>User</code> , or two separate people-tables?	One, with an optional link and no duplicated fields.
2. Vocabularies	Editable lookup tables, or frozen <code>choices</code> in code?	Tables by default — it is what preserves Palak's self-service.
3. Organisations	Do we add an institution entity, or keep affiliation as free text?	Add it — permission to publish is granted by institutions.
4. The public form (open since June)	Suggest-a-source tool, or general front door? And who rebuilds it, when?	Suggest-a-source, with a visible untriaged intake state.
5. Personal data	Lawful basis, consent, erasure path, and the encryption standard.	Settle before populating; match <code>users.User</code> .
6. Terminology	Rename Datasets → Gazetteers, with the nullable link carrying prospect vs actual?	Yes; label the print sense "printed gazetteer".
7. The registers	What do we call the three registers inside GRACE?	Catalogue · Engagement · Pipeline — "dashboard" is taken.

The system's own name is **settled: GRACE** — see §9.

Once 1, 2 and 3 are settled the models can be written, and the build divides cleanly: the schema and the admin are a developer job, while the vocabularies, the views and the day-to-day shape of the tracker stay Palak's — which is the arrangement Baserow was chosen to provide and which §5 is designed to preserve without it.